MachBoost: Conditional Acceleration for On-Device Text and Visual Models

Vistrit Pandey
vistrit@blinkfire.com

July 2026

Abstract

MachBoost is a local inference package for conditional acceleration. Its text path drafts from explicit local context and verifies candidate token blocks with the target model. For visual requests, it can either reuse state derived from unchanged image bytes or, experimentally, shorten a new image's visual sequence after Qwen3-VL completes its early cross-modal fusion. We evaluate these paths on an Apple M1 Max. Context-backed code continuation with a 3B MLX model reaches a 1.96× median paired speedup over native `mlx-lm`; copied retrieval answers reach 1.58× with experimental one-token re-entry. Across six Qwen vision models, repeated-image paired medians range from 5.14× to 17.44×. A separate Qwen3-VL 8B pilot uses ten unique TextVQA images with all caches disabled. Retaining a median 35.12% of visual hidden states after layer 3 yields a 1.67× aggregate wall-time speedup and a 1.70× median paired speedup. Native and compressed paths each match an accepted dataset answer on eight of ten questions, although normalized outputs agree on only seven. The first-view result is therefore approximate, single-model evidence rather than an exact or universal acceleration claim.

1 Introduction

Running LLMs on a laptop has become realistic, but it has not made autoregressive decoding disappear. A small model on Apple Silicon may fit in memory and still feel slow because the loop is serial: produce a token, update the prefix, run the model again. That bottleneck is especially visible in developer workflows, where the model often sits inside an editor, command runner, notebook, or local assistant.

Visual assistants add a different cost. One image may expand into roughly a thousand language-model input positions before the answer begins. Asking a second question about an unchanged screenshot, invoice, diagram, or camera frame can repeat both the vision encoder and the long visual-token prefill. A first question cannot use that cache, but it may still carry redundant visual states through every language layer.

Speculative decoding offers a way around part of this loop. A cheap source guesses several future tokens; the target model checks the guess; accepted tokens are emitted without paying for one full target step each. Prior work usually gets the draft from another model, extra heads, or a reference sequence. MachBoost takes the reference idea and applies it to the local machine. If the user is asking about a repo, config, note, policy, or retrieved passage, the likely continuation may already be present in that local text.

MachBoost does not try to accelerate every prompt or modality with one trick. It accelerates grounded text when a verifier can accept useful drafts, repeated-image queries when the visual

input has not changed, and selected Qwen3-VL first-view requests through an opt-in lossy path. Ordinary requests still follow the backend's native implementation.

This paper contributes the following:

- a context-only drafter and initial-candidate gate that avoid treating the prompt wrapper itself as external evidence;
- a cache-aware MLX verifier that fuses continuation and draft scoring, rewinds rejected cache entries in place, and resumes native decoding when useful context ends;
- a paired benchmark against native `mlx-lm` generation with raw rows, run-order alternation, environment provenance, and exact token comparisons;
- a resident local server with warm model lifecycle, streaming Ollama-compatible and OpenAI-compatible endpoints, and raw completion support for editor integrations;
- an architecture-aware MLX-VLM adapter with content-addressed image identity, projected-feature reuse where the model interface preserves all required tensors, and image-scoped prompt state;
- a whole-prefix checkpoint path for Qwen3.5's mixed recurrent/attention cache, plus a conservative prompt-state-only path for Qwen3-VL's deep-stack visual features;
- paired repeated-image artifacts for six Qwen models that record cache hits, matched prefix length, first-token latency, raw outputs, and expected-answer checks;
- an experimental Qwen3-VL wrapper that performs query-weighted spatial merging only after all early deep-stack visual injections, with per-request retention telemetry;
- a paired unique-image TextVQA pilot against the same model's full-token path, with both cache systems disabled and raw outputs retained;
- negative experiments with a small draft model, an MTP draft path, DFlash, an Apple Neural Engine draft, and lossless weight compression.

2 Background and Related Work

Speculative decoding was introduced as a way to accelerate autoregressive transformer inference without changing outputs by drafting candidate tokens and verifying them in parallel with the target model [9]. Speculative sampling independently developed a related draft-and-verify method for sampled decoding and reported 2–2.5× speedups in a distributed setting [5]. Subsequent work explored architecture-level or self-drafting variants. Medusa adds multiple decoding heads to predict future tokens and verifies candidate trees [4].

MachBoost is closest to methods that exploit references or prompt overlap rather than an auxiliary draft model. LLMA observes that outputs in retrieval and reference-grounded scenarios often share spans with the input reference, then verifies copied spans to preserve greedy output [21]. Prompt Lookup Decoding replaces the draft model with n-gram matching over the prompt [15]; PLD+ extends that idea by using language-model artifacts such as attention or hidden states to rank candidate spans [16]. Hugging Face Transformers exposes prompt lookup through `prompt_lookup_num_tokens` in assisted generation [7]. MachBoost differs mainly in packaging and operating assumptions: it treats local files and retrieved snippets as ordinary inputs to the

accelerator, keeps benchmark data machine-readable, and makes low-overlap prompts a measured fallback case rather than an error.

Results above 2× elsewhere generally require more than process tuning. ReDrafter trains a recurrent draft conditioned on target hidden state and reports up to 2.3× on Apple Silicon [22]. QuantSpec changes the draft's weight and KV-cache representation and reports speedups up to about 2.5× in long-context inference [17]. Mirror-SD maps complementary draft and target work across GPU and NPU devices and reports 2.8–5.8× on server-scale models [3]. At the runtime layer, BaseRT's native Metal kernels report up to 1.35× over MLX decode rather than a universal 2× [14]. These are different operating points: trained model changes, heterogeneous execution, or low-level kernels. MachBoost's current context path needs none of them, but only applies when the requested continuation overlaps local text.

The same distinction appears outside text generation. Diffusion workloads can reuse intermediate transformations across denoising steps; EasyCache reports 2.1–3.3× for several video models [24]. That mechanism does not transfer directly to autoregressive text decoding. It does, however, support a broader product design in which one package selects a backend-specific accelerator rather than pretending that a single algorithm fits text, image, audio, and video models.

Attention-state reuse is established. Prompt Cache stores reusable attention states for prompt modules and reports large first-token latency reductions on repeated prefixes [6]. MLX-VLM supplies prompt-state reuse, automatic prefix caching, and projected-feature caching [10]. MachBoost composes those facilities behind content identity and adds model-specific safety rules. It is not the origin of feature or prefix caching. The engineering question studied here is whether one local serving layer can apply them without silently dropping architecture-specific visual or recurrent state.

Visual token reduction is also an active field rather than a MachBoost invention. AdaptInfer uses dynamic text guidance and layer-dependent pruning [23]. Wang et al. report that visual-token usefulness changes with depth and task, describing an architecture- and workload-dependent "information horizon" [18]. ZOO-Prune estimates token sensitivity without training and reports up to 2.30× end-to-end acceleration while pruning as much as 94.4% of visual tokens [8]. The first-view mechanism studied here is simpler: it uses no calibration data, gradients, or model changes, and targets one MLX Qwen3-VL implementation. Its contribution is the post-deepstack integration and local paired evidence, not the general idea of dropping or merging visual tokens.

The implementation targets common local inference stacks. Hugging Face Transformers provides the broad model API surface for Python inference [19]. MLX is an Apple Silicon-oriented array framework and runtime used here for Mac-native evaluation [1]. Text measurements use a Qwen-family model converted for MLX; Qwen3 is the relevant public model-family report [20]. Visual measurements cover Qwen2.5-VL [2], Qwen3-VL [11], and Qwen3.5 [12].

3 Problem Setting

Let M be a target autoregressive model and let x be the prompt token sequence. Greedy decoding emits

$$y_t = \arg\max_v M(v \mid x, y_{<t})$$

for each generation step t. MachBoost receives a corpus C derived from prompt text, retrieved context, local files, or user-provided strings. The goal is to reduce wall-clock latency while returning the same sequence y that greedy decoding with M would return.

This paper stays with greedy decoding. Sampling-compatible speculative decoding is possible in principle, but greedy argmax verification gives a simple correctness check: the accelerated sequence

must match the serial baseline token-for-token.

For repeated visual queries, let I be an image and q_i a sequence of questions sent to one resident vision-language model V. With identical image bytes, the desired invariant is

$$\text{CachedVLM}(V, I, q_i) = \text{NativeVLM}(V, I, q_i)$$

under the same deterministic generation settings. In practice, cold full-prefill and warm short-suffix kernels can follow different floating-point reduction paths, so we record both literal equality and task-answer agreement. No reuse is permitted across distinct content identities.

The first-view path has a different contract. Let P_r compress visual hidden states to an approximate retention ratio r after early fusion. We measure

$$\text{CompressedVLM}(V, I, q, r) = V_{>k}(P_r(V_{\leq k}(I, q)))$$

against the unmodified full-token path for latency and task score. Equality is measured but not guaranteed: merging changes the language model's hidden sequence. This distinction prevents the exact cache result and the approximate first-view result from being combined into one quality claim.

4 Method

The implementation has a drafter, a verifier, and an escape hatch to native generation. The escape hatch is important: speculative decoding is a conditional optimization, not a replacement decoder.

4.1 Drafting from Nearby Text

The drafter indexes token n-grams from a corpus C. At generation time it looks at the current prefix $p = x \| \hat{y}$, searches for suffixes of p that occur in C, and proposes the tokens that followed the best matching span. Longer suffix matches are preferred, with draft length as a secondary signal. Only explicit context is indexed in the reported adaptive mode. An earlier implementation indexed the concatenation of prompt and context; because benchmark prompts already displayed the source passage, that design could draft from the prompt wrapper and blur the boundary between available context and generated history.

Before entering the verifier loop, MachBoost asks whether the generation boundary has a context candidate. If not, the default profile calls native `mlx-lm` generation immediately. An opt-in re-entry profile generates a bounded number of target tokens, checks the context index again, and either enters block verification or resumes native generation from the live cache. Re-entry can recover copied answers whose first token begins a span in retrieved text. It also changes the cache trajectory, so version 0.2.0 disables it by default.

4.2 Verification Against the Target Model

Given a prefix p and draft $d = (d_1, \ldots, d_k)$, the verifier checks how many draft tokens equal the target model's greedy continuation. A verifier-capable runtime scores a block in one target call and compares target argmax tokens at the relevant positions. If the first a draft tokens are accepted, they are committed. On a mismatch, the target's residual token is emitted. When the whole block is accepted, the verifier also returns the next target token already present in the logits. The following round scores that pending token together with the next draft, avoiding a separate target call between accepted blocks.

Under deterministic target arithmetic, the intended invariant is:

$$\mathrm{Boost}(M, x, C, T) = \mathrm{Greedy}(M, x, T)$$

for a maximum generation length T. Batched and scalar floating-point paths can still choose different argmax tokens at a near tie, so this is tested rather than assumed. Every benchmark row records an `output_match` flag from the native and accelerated token IDs.

4.3 Keeping the Cache Path Stable

Verification writes candidate entries to the MLX prompt cache. Accepted entries remain useful; rejected entries must not survive. MachBoost trims the rejected suffix in place when the cache type permits it. If safe trimming is unavailable, the service reconstructs the accepted prefix. Prompt initialization follows `mlx-lm`'s split prefill trajectory, processing the final prompt token separately. Once no further context candidate exists, the service hands the live verifier cache to `mlx_lm.generate_step`; the remaining response therefore uses the runtime's optimized asynchronous decoder instead of a Python-level `argmax`/synchronize loop.

4.4 Architecture-Aware Reuse for an Unchanged Image

The visual adapter computes a SHA-256 identity from the image contents. Local file metadata memoizes hashing but does not define identity; base64 payloads, data URLs, and in-memory images are hashed directly. The first bounded LRU maps this identity to the projected output of the vision tower,

$$h(I) \longmapsto E_V(I).$$

On a miss, the vision tower processes the pixel tensor and image grid. On a hit, the stored features enter through `cached_image_features`. Qwen2.5-VL and Qwen3.5 can use this path. Qwen3-VL also emits deep-stack visual tensors; retaining only its primary projected tensor is incomplete, so MachBoost disables feature-only reuse for that family.

Feature reuse still leaves hundreds of visual positions for the language model to prefill. A second bounded map associates $h(I)$ with reusable prompt state. Pure attention models can retain the matching KV prefix. Qwen3.5 is different: its language model interleaves ordinary KV layers with recurrent linear-attention arrays. Rewinding only keys and values leaves the recurrent arrays at the wrong point. MachBoost uses MLX-VLM's exact checkpoint facility for that family, storing a complete cache snapshot shortly before the question suffix and restoring both cache types together. A preliminary KV-only implementation changed answers and was discarded.

All state belongs to one loaded model instance and is cleared on reset or unload. This avoids cross-model tensor reuse and makes image replacement an ordinary miss. The method does not alter parameters or generated-token decoding.

4.5 Post-Fusion Compression for a New Image

Qwen3-VL supplies a primary visual sequence and additional deep-stack visual embeddings to its language model. Removing tokens before those injections changes their alignment, so MachBoost first executes every configured deep-stack injection. The current model uses three such layers; compression therefore occurs after layer 3 and before the remaining 33 language layers.

For each contiguous image segment, visual positions are grouped by temporal index and a spatial block size derived from the requested retention ratio. The mean hidden state of up to 48 trailing text positions forms a query vector. Within each spatial group, cosine similarity to this query is

converted to a softmax weight and the weighted visual states are merged into one representative. Adaptive mode ranks groups by internal directional dispersion and leaves the most diverse groups unmerged until the requested visual-token budget is met. Text positions are never removed.

The wrapper updates position identifiers and attention masks to the retained sequence. It records original and retained sequence lengths, visual-token counts, layer index, and actual retention ratio. Requests using this path bypass prompt and visual caches so the measured behavior cannot depend on an earlier image. The implementation delegates to the original MLX-VLM model unchanged when the option is off, the request is not batch one, or no visual positions are present.

4.6 Deciding When to Use the Drafter

Drafting from local text is sometimes the wrong choice. Open-ended prompts often accept only a few draft tokens, so verification overhead dominates. MachBoost therefore exposes a benchmark step. For a prompt family, it measures:

- token match between baseline and boosted output,
- wall-clock speedup,
- accepted draft tokens divided by generated tokens,
- target-call or forward-call reduction.

The high-level API can also calibrate an accelerator across representative prompts:

```
from machboost import Accelerator, GatePolicy

boost = Accelerator.from_mlx(model, context_paths=["./docs", "./src"])
calibration = boost.calibrate(
    prompts,
    max_tokens=32,
    gate_policy=GatePolicy(min_speedup=1.05,
                           min_acceptance_rate=0.10),
)
text, stats = boost.generate(prompt, max_tokens=128)
```

If the measured workload fails the policy, `generate` uses the native backend path. Calibration and the per-request initial-candidate gate serve different purposes: calibration enables a workload family, while the gate prevents verifier overhead on an individual request with no draft at its boundary.

5 Implementation

MachBoost is implemented as a small Python package with no required runtime dependencies. Optional extras install backend-specific dependencies:

- `machboost[mlx]` for MLX and `mlx-lm`,
- `machboost[hf]` for PyTorch and Transformers,
- `machboost[vision]` for MLX-VLM,

- Ollama HTTP support for benchmarking and capability detection.

The core package defines a `StepService` protocol with `next_token(prefix)` and an optional `verify(prefix, candidate)` hook. A service with only `next_token` can use the wrapper but cannot skip target work. A verifier can accept several draft tokens per target call. The high-level `Accelerator` loads MLX or Hugging Face models, reads strings, files, or directories as context, and exposes `benchmark`, `calibrate`, and `generate`.

Version 0.2.0 added a resident HTTP process. It owns loaded accelerators, serializes requests per model, retains each model indefinitely by default, and supports finite idle lifetimes or explicit unloading. The CLI can start this process automatically, preload a short model alias such as `qwen2.5:3b`, and issue chat or raw completion requests without reloading weights. The server exposes `/api/chat`, `/api/generate`, model lifecycle endpoints, and OpenAI-compatible chat and completion endpoints. Native MLX streaming forwards `mlx-lm`'s already-detokenized text segments; verified multi-token spans use a stateful streaming detokenizer. This avoids the earlier cost of decoding the entire token prefix once per emitted token.

The 0.3 series adds vision-model aliases, image arguments in the CLI and Python client, Ollama-style image payloads, and OpenAI-style image content parts. The current alias set includes Qwen2.5-VL 3B, Qwen3-VL 2B/4B/8B, and Qwen3.5 0.8B/4B/9B. MLX objects are created and used on one dedicated worker thread because MLX arrays and streams are thread-affine. The worker owns model load, image preparation, cache lookup, generation, and cache release. Version 0.4 adds the opt-in `vision_tokens` mode and retention telemetry. It is restricted to Qwen3-VL because the wrapper depends on that family's layer and deep-stack interfaces.

The Ollama HTTP adapter remains a wrapper: Ollama's public generation API does not expose the target token IDs, logits, mutable KV cache, and block-verification hook this implementation needs. Native integration would require a runner-level change rather than an HTTP option.

6 Evaluation

6.1 Machine and Model

Measurements were collected on an Apple M1 Max with 10 CPU cores and 32 GB unified memory, running macOS 26.5.2. The environment used Python 3.13.3, MLX 0.31.2, `mlx-lm` 0.31.3, and MLX-VLM 0.5.0. Text experiments used `mlx-community/Qwen2.5-3B-Instruct-4bit`. The repeated-image matrix used MLX 4-bit conversions of Qwen3-VL 2B/4B/8B and Qwen3.5 0.8B/4B/9B. The first-view pilot used `mlx-community/Qwen3-VL-8B-Instruct-4bit`. Only one model was resident during each run. Model fetch and load time were recorded but excluded from paired request latency.

The benchmark requests 64 greedy tokens for three fixture families: literal Python continuation, retrieval-grounded answering, and open-ended writing. Each family is repeated five times with a fresh nonce under both default and re-entry profiles. Baseline and accelerated order alternate. Wall time includes prompt processing. The baseline calls `mlx-lm`'s native streaming generator; it is not MachBoost's `next_token` loop. The artifacts `results/mlx_native_default_qwen25_3b_20260713.json` and `results/mlx_native_reentry_qwen25_3b_20260713.json` contain prompts, token previews, raw times, package versions, hardware data, and each run's order.

6.2 Native-Baseline Results

Profile/path	Match	Median	Base tok/s	MB tok/s	Accepted
Default/code	100%	1.96×	87.46	167.28	51
Default/RAG	100%	1.04×	89.98	91.61	0
Default/open	100%	1.00×	99.95	98.27	0
Re-entry/code	100%	1.62×	72.54	121.38	51
Re-entry/RAG	100%	1.58×	88.92	140.09	30
Re-entry/open	100%	1.08×	94.25	101.57	0

Table 1: Five fresh-nonce repeats per profile and fixture. Speedup is paired wall time; throughput and accepted-token columns are per-column medians. "MB" denotes MachBoost.

The default code fixture begins at a boundary that occurs in the supplied source. It accepts a median 51 of 64 output tokens and reduces logical target forwards by 76.6%. Its paired speedups are 2.44, 1.15, 2.08, 1.96, and 1.91×. The 1.96× median is close to the requested 2× threshold but does not clear it, and the range shows how noisy short local measurements remain.

The default retrieval answer has no candidate at its generation boundary and takes native fallback. With one-token re-entry and 1-gram lookup, the first target token anchors a context span; the verifier then accepts a median 30 tokens and reaches 1.58×. Open-ended generation accepts no draft tokens. Its apparent 1.08× ratio is timing noise rather than an algorithmic gain.

All 30 reported rows match their paired native token sequences. A separate 15-row exploratory run with a three-token re-entry probe had one RAG mismatch after an accepted span. This is why re-entry remains opt-in. For a user-facing system, the useful statistics are coverage, conditional speedup, and equality rate rather than one aggregate median.

6.3 Resident Serving Results

The resident benchmark preloads the same 3B model, then sends five forced 64-token completions and five short streaming chats. It measures wall time at the server or Python streaming client, including prompt processing and detokenization. Table 2 summarizes the artifact `results/resident_qwen25_3b_20260713.json`.

Workload	Rows	Median TTFT	Median total	End-to-end tok/s
Forced 64-token completion	5	–	0.657 s	97.47
Short streaming chat	5	0.298 s	0.358 s	–

Table 2: Resident-server latency after preloading the model. Chat outputs contain nine tokens in each recorded row; TTFT is measured when the client receives the first non-empty text segment.

The first forced completion takes 0.973 seconds because the Metal execution path still needs initialization after weight loading; the following four take 0.650–0.663 seconds. These rows accept no context draft tokens. The measurement therefore demonstrates load amortization, corrected streaming, and native fallback rather than a speculative-decoding gain. A same-day Ollama probe records a 0.883-second median for the same forced-token shape, a 1.35× wall-time ratio relative to the 0.657-second resident run. The two runtimes use different 4-bit model formats and the requests were not interleaved, so this is a backend-suitability observation rather than exact-weight evidence.

6.4 Repeated-Image Results

The visual benchmark uses a generated 1024 by 768 image containing a project name, status, budget, owner, and labeled colored shape. Four short extraction questions are each repeated three times. Every pair sends one request with both visual caches disabled and one request with both enabled; pair order alternates by repetition. Both paths use the same loaded model, greedy decoding, a 16-token limit, and the same image. Before recording, the harness performs one uncached warm-up and one cached prime. Table 3 summarizes `results/vision_cache_qwen25_3b_20260714.json`.

Metric	Uncached	Accelerated	Ratio
Median wall time	2.818 s	0.152 s	18.58×
Median paired wall time	–	–	18.33×
Median time to first text	2.807 s	0.144 s	19.45×
Effective prompt throughput	379.43 tok/s	12928.44 tok/s	34.07×

Table 3: Twelve uncached and 12 accelerated requests over one unchanged image. Effective prompt throughput divides the full logical prompt length by measured prefill time even when cached tokens are not recomputed.

The median of per-pair wall-time ratios is 18.33×; individual ratios are retained in the artifact. All 12 accelerated outputs exactly equal their paired uncached outputs, and both modes answer every fixture question correctly. Projected features hit in all recorded accelerated rows. Eleven rows reuse a median 1,018-token visual prefix and range from 13.32× to 21.36×. The feature-only control reaches 1.33×.

The distinction between those rows explains most of the result. Skipping the vision tower alone removes useful work, but the language model still processes the visual token span. Reusing its prompt state reduces the second stage to the changed question suffix. Generated answers are only two to five tokens long, so time to first text dominates total latency. Generation throughput after the first token is not improved by this mechanism.

The broader matrix uses the same image, questions, generation settings, warm-up policy, and alternating pair order. Table 4 summarizes `results/vision_cache_qwen_matrix_20260714.json`; each row contains 12 pairs.

Model	Total	Base	Cached	Paired	TTFT	Exact
Qwen3-VL 2B	2B	1.524 s	0.132 s	11.41×	12.23×	100%
Qwen3-VL 4B	4B	2.743 s	0.214 s	12.73×	13.32×	100%
Qwen3-VL 8B	9B	5.152 s	0.307 s	16.69×	17.30×	100%
Qwen3.5 0.8B	0.9B	0.656 s	0.125 s	5.14×	5.48×	75%
Qwen3.5 4B	5B	3.199 s	0.203 s	14.29×	16.43×	100%
Qwen3.5 9B	10B	5.572 s	0.311 s	17.44×	18.82×	100%

Table 4: Cross-model repeated-image results. "Paired" is the median of per-pair wall-time ratios. "Total" follows the official multimodal model listings rather than the variant label or quantized file size. Every baseline and cached row returns the expected fixture answer.

The median of the six model-level paired medians is 13.51×. All 144 measured responses return the expected answer. Literal equality holds in 69 of 72 pairs (95.83%); the three differences come from Qwen3.5 0.8B placing or omitting a semicolon around the same `BLUE SQUARE` answer inside a JSON fence. Qwen3-VL records no projected-feature hits, by design, and its three genuine no-prefix controls have a 0.99× median. Its remaining 33 cached rows reuse a 776-token prefix. All 36

Qwen3.5 cached rows restore a 776-token whole-state checkpoint.

The official Qwen3.6 collection lists a 27B variant with 28B total parameters and a 35B-A3B variant with 36B total parameters [13]. Neither meets the requested small-model constraint, even though the latter activates 3B parameters per token and quantized files can occupy less than 10 GB. Qwen3-VL 8B is listed as 9B total, while Qwen3.5 9B is listed as 10B total; the table keeps these boundary cases explicit.

6.5 Unique-Image Post-Fusion Pilot

The first-view harness draws ten unique image/question pairs from the public TextVQA dataset. Each pair compares the native full-token path with adaptive post-fusion compression at a requested 35% visual-token ratio. Both visual and prompt caches are disabled. Pair order alternates, and one held-out unique image warms each mode before recording. Generation is greedy with a 16-token limit. The artifact `results/cold_vision_qwen3vl_8b_postfusion_20260715.json` stores source identifiers, accepted dataset answers, raw model outputs, timing, cache counters, and retained-token telemetry.

Metric	Native	Post-fusion	Ratio/agreement
Median wall time	4.078 s	2.368 s	1.72×
Aggregate wall time	43.771 s	26.170 s	1.67×
Median paired wall time	–	–	1.70×
Median time to first text	4.029 s	2.351 s	1.71×
Accepted-answer match	80%	80%	unchanged
Normalized output equality	–	–	70%

Table 5: Ten unique TextVQA pairs on Qwen3-VL 8B. The accelerated path retains a median 35.12% of visual hidden states after layer 3. Ratios include vision encoding, language prefill, and short-answer decode, but exclude model load.

The median paired result is 1.70× and the ratio of total recorded times is 1.67×. Time to first text accounts for nearly all of the reduction, as expected for answers capped at 16 tokens. No row reports a visual-cache hit or reused prompt prefix. Native and compressed paths each match an accepted TextVQA answer on eight rows, but they are not token-equivalent: normalized outputs match on seven rows and literal outputs on five. Equal aggregate task score on this small sample does not show that errors are identical or that quality is preserved on the dataset as a whole.

6.6 Why Earlier Ratios Were Rejected

Earlier repository artifacts reported 3–8× on MLX. Their baseline disabled the prompt cache or performed a synchronized target call for each token. That path ran near 10 tokens/s and was not representative of native `mlx-lm`, which uses cached asynchronous generation. Those artifacts remain available as implementation diagnostics, but they cannot support a production speed claim. Replacing that baseline was also what exposed a regression in MachBoost's serial tail; switching the tail to native generation restored parity for non-overlap prompts.

A one-repeat Hugging Face artifact likewise found MachBoost context lookup competitive with Transformers prompt lookup, but it predates the current paired harness. It is useful for designing a larger comparison, not for the headline result.

6.7 Exploratory Alternatives

We tested several ways to extend acceleration beyond literal context overlap. Table 6 summarizes the local probes. They were not all run under one benchmark protocol, so the numbers are diagnostic rather than comparative.

Path	Local result	Main constraint
DFlash, Qwen3.5 9B	0.66× at 128; 0.96× at 512	M1 lacks the newer accelerator path; draft emulation costs more than accepted work saves.
Qwen2.5 0.5B draft for 3B target	0.38–0.78×	Acceptance of roughly 36–58% did not repay serial draft generation.
Ollama embedded MTP	12.7–24.3 tok/s versus roughly 40 tok/s baseline	The tested draft depths added overhead on this model and machine.
ANE 0.5B draft	51.1 decode tok/s	A one-token ANE draft cannot feed a 3B MLX target near 102 tok/s economically; multi-token heads are needed.
Lossless Q4 compression	1.08–1.15× ideal entropy bound	The quantized weights are already close to high entropy; decoder cost would reduce the gain further.

Table 6: Exploratory paths that did not beat native generation locally. Each result is specific to the tested model, implementation, and M1 Max.

The failures share a simple accounting problem. A draft only helps when the time removed from target decoding exceeds draft generation, verification, synchronization, and cache repair. A smaller model is not automatically a cheap enough draft. Moving it to another processor is not sufficient either if it predicts one token at a time. The strongest published results solve this with trained multi-token prediction, high acceptance, or concurrent heterogeneous execution rather than with process priority.

7 Limitations

The text evidence has 30 headline rows, one model, one quantization, one machine, and controlled 64-token prompts. The resident evidence adds ten requests but does not broaden model or hardware coverage. The accelerated code fixture is intentionally favorable: the desired code is substantially present in context. It demonstrates the mechanism but does not estimate how often real users encounter an eligible boundary. Longer repository sessions, multiple 3B–9B architectures, prefill-heavy prompts, concurrent clients, and energy measurements remain untested under the corrected harness.

The repeated-image evidence spans six model sizes but remains one synthetic image, four extraction questions, three repeats, short answers, two closely related Qwen families, and one Apple Silicon machine. It excludes model load and starts after an explicit prime. The result says nothing about changed images, long-form visual reasoning, memory pressure under many cached images, or concurrent sessions. Qwen3-VL's no-prefix cache controls show no gain from cache machinery alone. Since decode work is unchanged, the ratio should shrink as answer length grows.

The first-view evidence is smaller: ten OCR-heavy TextVQA rows, one quantized Qwen3-VL 8B model, one retention ratio, one layer boundary, and one machine. It has neither exact output preservation nor enough samples for a quality-equivalence or statistical significance claim. We

have not compared the implementation directly with AdaptInfer, ZOO-Prune, random pruning, attention ranking, or an optimized CUDA implementation. The 35% policy is consequently an experimental profile, not a generally selected operating point. Audio and video do not yet have MachBoost adapters.

Only greedy decoding is evaluated. Even then, cold and warm kernels can use different reduction shapes; the Qwen3.5 0.8B punctuation drift is a concrete example of literal non-invariance without a task-answer change. This does not establish equivalence for sampling or beam search. Ollama is also not accelerated through its public HTTP API.

Finally, this work introduces neither prompt lookup, prompt caching, nor visual-token pruning. The contribution is a set of local integrations, safety boundaries, and paired artifacts: context drafting, architecture-aware cache handling, and post-deepstack Qwen3-VL compression. A stronger research claim needs broader comparisons with LLMA, Prompt Lookup Decoding, PLD+, MLX-VLM's current server, production prefix-cache systems, and established VLM pruning baselines.

8 Next Experiments

There are two plausible tracks. The first keeps models unchanged: add an online eligibility predictor, extend bounded re-entry beyond the current one-token experiment, and add native verifier hooks for llama.cpp/Ollama. The resident server makes this testable under long sessions; evaluation should report request coverage, cold and warm time to first token, decode throughput, peak memory, concurrency, energy, and exact outputs against each backend's own optimized generator.

The second track accepts model-specific preparation. A multi-token drafter distilled from the target could run on the Apple Neural Engine while target verification runs on the GPU, following the heterogeneous lesson of Mirror-SD. Quantized draft weights and KV state could reduce memory traffic in long contexts. Both ideas exceed the scope of a wrapper and should be treated as separate research prototypes with quality and compatibility tests.

The immediate visual work is external validity. The first-view harness should expand from TextVQA to chart, document, scene, spatial-reasoning, and hallucination-sensitive datasets; add non-Qwen VLMs; sweep layer, retention ratio, image size, and answer length; and report confidence intervals, memory, energy, and task metrics. Comparisons with random removal, attention ranking, AdaptInfer, and ZOO-Prune are needed to determine whether query-weighted merging contributes beyond token count alone. An online policy should remain disabled unless a model/task calibration set meets both latency and quality thresholds. Video should be treated separately through temporal change detection or frame-token reduction, with video-specific quality tests.

9 Conclusion

MachBoost now exposes three conditional paths. On a 3B text model, literal context-backed code continuation reaches a 1.96× median over five paired runs; one-token re-entry reaches 1.58× on copied retrieval answers. Across six visual models, short repeated questions over one unchanged image produce model-level paired medians from 5.14× to 17.44×. For a new image, post-fusion Qwen3-VL compression reaches 1.67× aggregate speedup on a ten-row TextVQA pilot while both modes score 8/10. That path is approximate: normalized output equality is 70%, and the experiment is too small to establish quality parity. The useful result is therefore narrower than universal faster inference. Context overlap can reduce text decode, unchanged content can remove repeated

visual prefill, and post-fusion token reduction can lower first-view prefill for a tested Qwen3-VL workload. Each mechanism has a separate eligibility and correctness contract.

References

[1] Apple Machine Learning Research. Mlx: An array framework for apple silicon. https://github.com/ml-explore/mlx, 2023. Software.

[2] Shuai Bai, Keqin Chen, Xuejing Liu, Jialin Wang, Wenbin Ge, Sibo Song, Kai Dang, Peng Wang, Shijie Wang, Jun Tang, Humen Zhong, Yuanzhi Zhu, Mingkun Yang, Zhaohai Li, Jianqiang Wan, Pengfei Wang, Wei Ding, Zheren Fu, Yiheng Xu, Jiabo Ye, Xi Zhang, Tianbao Xie, Zesen Cheng, Hang Zhang, Zhibo Yang, Haiyang Xu, and Junyang Lin. Qwen2.5-VL technical report. *arXiv preprint arXiv:2502.13923*, 2025.

[3] Nikhil Bhendawade, Kumari Nishu, Arnav Kundu, Chris Bartels, Minsik Cho, and Irina Belousova. Mirror speculative decoding: Breaking the serial barrier in llm inference. Apple Machine Learning Research, 2025.

[4] Tianle Cai, Yuhong Li, Zhengyang Geng, Hongwu Peng, Jason D. Lee, Deming Chen, and Tri Dao. Medusa: Simple llm inference acceleration framework with multiple decoding heads. *arXiv preprint arXiv:2401.10774*, 2024.

[5] Charlie Chen, Sebastian Borgeaud, Geoffrey Irving, Jean-Baptiste Lespiau, Laurent Sifre, and John Jumper. Accelerating large language model decoding with speculative sampling. *arXiv preprint arXiv:2302.01318*, 2023.

[6] In Gim, Guojun Chen, Seung-seob Lee, Nikhil Sarda, Anurag Khandelwal, and Lin Zhong. Prompt cache: Modular attention reuse for low-latency inference. *arXiv preprint arXiv:2311.04934*, 2023.

[7] Hugging Face. Assisted decoding. https://huggingface.co/docs/transformers/en/assisted_decoding, 2026. Accessed July 6, 2026.

[8] Youngeun Kim, Youjia Zhang, Huiling Liu, Aecheon Jung, Sunwoo Lee, and Sungeun Hong. ZOO-Prune: Training-free token pruning via zeroth-order gradient estimation in vision-language models. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, pages 39572–39582, 2026.

[9] Yaniv Leviathan, Matan Kalman, and Yossi Matias. Fast inference from transformers via speculative decoding. In *Proceedings of the 40th International Conference on Machine Learning*, volume 202 of *Proceedings of Machine Learning Research*, pages 19274–19286. PMLR, 2023.

[10] MLX-VLM contributors. MLX-VLM: Vision language models on apple silicon. https://github.com/Blaizzy/mlx-vlm, 2026. Software, accessed July 14, 2026.

[11] Qwen Team. Qwen3-VL model collection. https://huggingface.co/collections/Qwen/qwen3-vl, 2026. Accessed July 15, 2026.

[12] Qwen Team. Qwen3.5 model collection. https://huggingface.co/collections/Qwen/qwen35, 2026. Accessed July 15, 2026.

[13] Qwen Team. Qwen3.6 model collection. https://huggingface.co/collections/Qwen/qwen36, 2026. Accessed July 15, 2026.

[14] Prabod Rathnayaka, Fabian Waschkowski, and Lukas Wesemann. BaseRT: Best-in-class LLM inference on apple silicon via native metal. *arXiv preprint arXiv:2607.00501*, 2026.

[15] Apoorv Saxena. Prompt lookup decoding. https://github.com/apoorvumang/prompt-lookup-decoding, 2023. GitHub repository.

[16] Shwetha Somasundaram, Anirudh Phukan, and Apoorv Saxena. Pld+: Accelerating llm inference by leveraging language model artifacts. *arXiv preprint arXiv:2412.01447*, 2024.

[17] Rishabh Tiwari, Haocheng Xi, Aditya Tomar, Coleman Hooper, Sehoon Kim, Maxwell Horton, Mahyar Najibi, Michael W. Mahoney, Kurt Keutzer, and Amir Gholami. Quantspec: Self-speculative decoding with hierarchical quantized kv cache. In *Proceedings of the 42nd International Conference on Machine Learning*, 2025.

[18] Yahong Wang, Juncheng Wu, Zhangkai Ni, Longzhen Yang, Yihang Liu, Chengmei Yang, Ying Wen, Lianghua He, Xianfeng Tang, Hui Liu, and Yuyin Zhou. When token pruning is worse than random: Understanding visual token information in VLLMs. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, pages 31910–31919, 2026.

[19] Thomas Wolf, Lysandre Debut, Victor Sanh, Julien Chaumond, Clement Delangue, Anthony Moi, Pierric Cistac, Tim Rault, Remi Louf, Morgan Funtowicz, Joe Davison, Sam Shleifer, Patrick von Platen, Clara Ma, Yacine Jernite, Julien Plu, Canwen Xu, Teven Le Scao, Sylvain Gugger, Mariama Drame, Quentin Lhoest, and Alexander M. Rush. Transformers: State-of-the-art natural language processing. In *Proceedings of the 2020 Conference on Empirical Methods in Natural Language Processing: System Demonstrations*, pages 38–45. Association for Computational Linguistics, 2020.

[20] An Yang, Anfeng Li, Baosong Yang, Beichen Zhang, Binyuan Hui, Bo Zheng, Bowen Yu, Chang Gao, Chengen Huang, Chenxu Lv, Chujie Zheng, Dayiheng Liu, Fan Zhou, Fei Huang, Feng Hu, Hao Ge, Haoran Wei, Huan Lin, Jialong Tang, Jian Yang, Jianhong Tu, Jianwei Zhang, Jianxin Yang, Jiaxi Yang, Jing Zhou, Jingren Zhou, Junyang Lin, Kai Dang, Keqin Bao, Kexin Yang, Le Yu, Lianghao Deng, Mei Li, Mingfeng Xue, Mingze Li, Pei Zhang, Peng Wang, Qin Zhu, Rui Men, Ruize Gao, Shixuan Liu, Shuang Luo, Tianhao Li, Tianyi Tang, Wenbiao Yin, Xingzhang Ren, Xinyu Wang, Xinyu Zhang, Xuancheng Ren, Yang Fan, Yang Su, Yichang Zhang, Yinger Zhang, Yu Wan, Yuqiong Liu, Zekun Wang, Zeyu Cui, Zhenru Zhang, Zhipeng Zhou, and Zihan Qiu. Qwen3 technical report. *arXiv preprint arXiv:2505.09388*, 2025.

[21] Nan Yang, Tao Ge, Liang Wang, Binxing Jiao, Daxin Jiang, Linjun Yang, Rangan Majumder, and Furu Wei. Inference with reference: Lossless acceleration of large language models. *arXiv preprint arXiv:2304.04487*, 2023.

[22] Aonan Zhang, Ray Zhang, Yunfei Cheng, Chong Wang, and Yi Wang. Recurrent drafter for fast speculative decoding in large language models. *arXiv preprint arXiv:2403.09919*, 2024.

[23] Weichen Zhang, Zhui Zhu, Ningbo Li, Kebin Liu, and Yunhao Liu. AdaptInfer: Adaptive token pruning for vision-language model inference with dynamical text guidance. *arXiv preprint arXiv:2508.06084*, 2025.

[24] Xin Zhou, Dingkang Liang, Kaijin Chen, Tianrui Feng, Xiwu Chen, Hongkai Lin, Yikang Ding, Feiyang Tan, Hengshuang Zhao, and Xiang Bai. Less is enough: Training-free video diffusion acceleration via runtime-adaptive caching. *arXiv preprint arXiv:2507.02860*, 2025.